Building and Securing a REST API

MoMo SMS Data Processing System

Week 3 Technical Report

1. Introduction to API security

The MoMo SMS data processing system gives a clear and compiled example of financial transaction data through a REST API. And because of the kind of information this data contain, such as sensitive information like real payment amounts, senders, receivers, and account balances. We must make API security a high priority. Without security controls, anyone who knows the server address could read, modify, or delete thousands of real transaction records.

That is why we have implemented Basic Authentication for this API as its security mechanism. This Basic Authentication requires the client to send a username and password with every request. The credentials are encoded in Base64 and included in the Authorization header of the HTTP request.

For example, when a client sends:

curl -u admin:MOMO123 http://localhost:8000/transactions

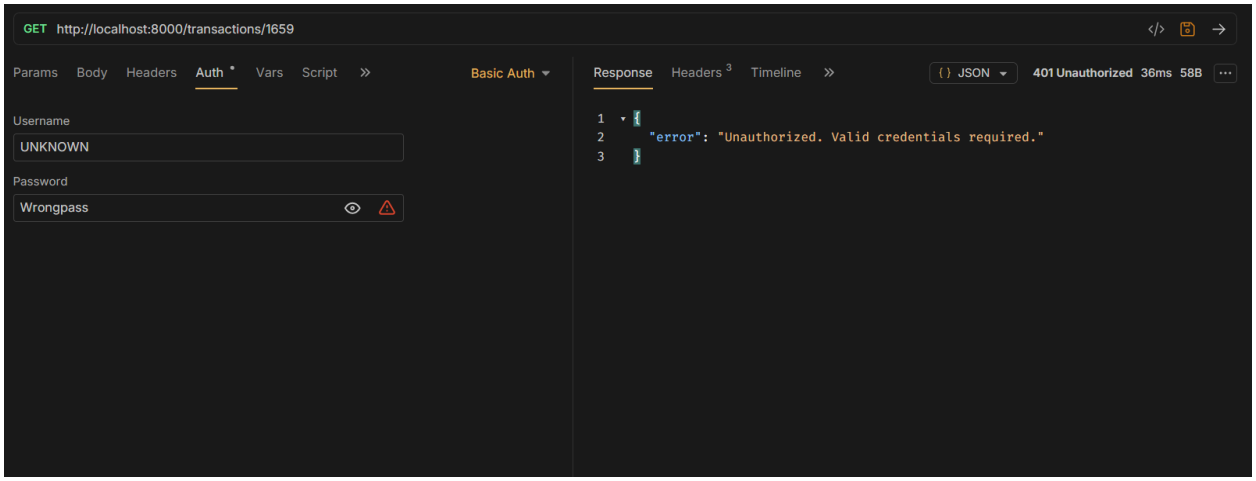
The browser or curl tool encodes the credentials as Base64 and sends the header:

Authorization: Basic YWRtaW46TU9NTzEyMw==

The server decodes this, draws out the username and password, and checks them against the stored credentials. If they match, the request is allowed. If they do not match, the server returns a 401 Unauthorized response and the request gets rejected.

i. How Authentication is Implemented

Before handling a request, each API endpoint checks the user's credentials by calling the check_auth() function. This function goes through the Authorization header, decodes the Base64 value, extracts the username and password, and compares them with the stored credentials. If the header is missing, invalid, or the credentials do not match, the function returns False and the endpoint immediately responds with a 401 response without executing any database logic.



2. Basic Auth Limitations and Stronger Alternatives

i. Why Basic Authentication is Weak

Basic Authentication has several serious limitations that make it unfit for a production level system handling real financial data.

- Credentials sent with every request: The username and password are included in every API request. Although credentials are encoded using Base64, it is not a proper encryption and can easily be reversed. If an attacker intercepts the request they may be able to decode it, retrieve the credentials, and gain unauthorized access.

- No expiry: Basic Authentication credentials do not expire automatically. If the password is leaked, it remains valid forever if not manually changed.
- No token revocation: Basic Authentication does not support session or token revocation. If an attacker gains access to the credentials, there is no way to disable only that access. The only option is changing the password for everyone.
- Hardcoded credentials : In the current implementation the username and password are stored directly in app.py file. This means that anyone who can access the source code can read these credentials, creating a security risk.
- No user identity: Basic Auth only checks a single username/password pair. This makes it unlikely to implement authorization with different users having different permissions.

ii. Stronger Alternatives

JSON Web Tokens (JWT)

JWT is a token-based authentication system. Instead of sending credentials with every request, the client logs in once and receives a signed token. This token then will be included in subsequent requests, typically in the header. Tokens have an expiring time, meaning that in a set amount of time it will be invalidated and require reauthenticating. JWTs are signed hence the server can verify authenticity of the token and depending on implementation the server can also invalidate tokens without changing any passwords using mechanisms such as token blacklist and rotating signing keys.

OAuth2

OAuth2 is the industry standard for authorization used by Google, GitHub, and most major APIs. It separates authentication from authorization. It allows

a user to grant an application access to their certain data without having to share their password with that application. OAuth2 supports multiple grant types for different scenarios: mobile apps, web apps, and server-to-server communication. For a MoMo financial API used by mobile money clients, OAuth2 would be the appropriate choice as it provides secure access while protecting user's credentials.

API Keys with HTTPS

A simpler alternative to Basic Authentication is using long randomly generated API keys. These keys are sent over HTTPS, which will encrypt the data being transmitted and helps protect the key from being intercepted. Such API keys can be revoked or replaced easily without affecting other users. This is a common approach for server-to-server communication where OAuth2 is overly complex.

Method	**Expiry**	**Revocable**	**Encrypted**	**Suitable For**
Basic Auth	Never	No	No (Base64 only)	Development and testing only
API Keys + HTTPS	Manual	Yes	Yes (HTTPS)	Server-to-server communication
JWT	Built-in	Yes	Yes	Web and mobile applications
OAuth2	Built-in	Yes	Yes	Production financial APIs

3. API Endpoint Documentation

The MoMo SMS API exposes five CRUD endpoints over HTTP. All endpoints require Basic Authentication. The base URL for local development is http://localhost:8000.

Testing API Endpoint

The endpoints were tested using Bruno, an open-source API client as evidenced by the screenshots.

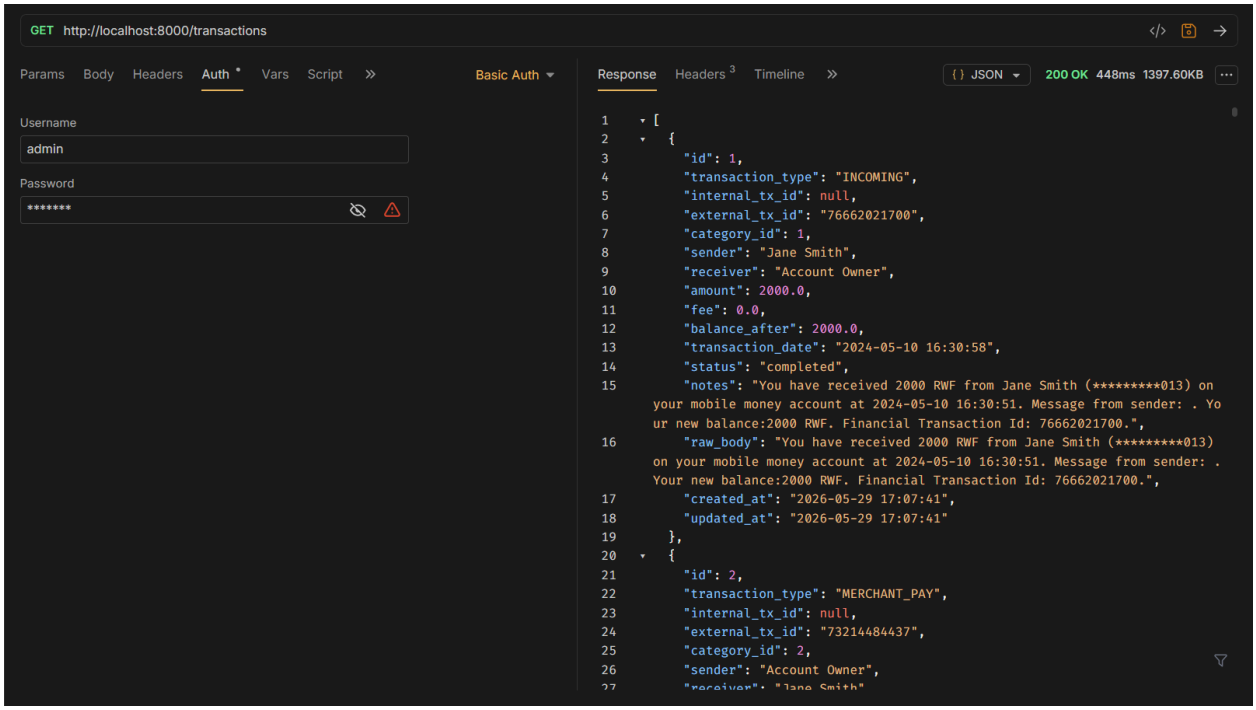
The “curl” is being used on the documentation for its easy access through the terminal, and doesn’t need a person to install third-party applications.

Method	**Endpoint**	**Description**
GET	/transactions	Retrieve all transactions from the database
GET	/transactions/{id}	Retrieve a single transaction by its ID
POST	/transactions	Create a new transaction record
PUT	/transactions/{id}	Update an existing transaction by ID
DELETE	/transactions/{id}	Permanently delete a transaction by ID

i. GET /transactions

Returns all transactions stored in the database as a JSON array.

curl -u admin:MOMO123 http://localhost:8000/transactions

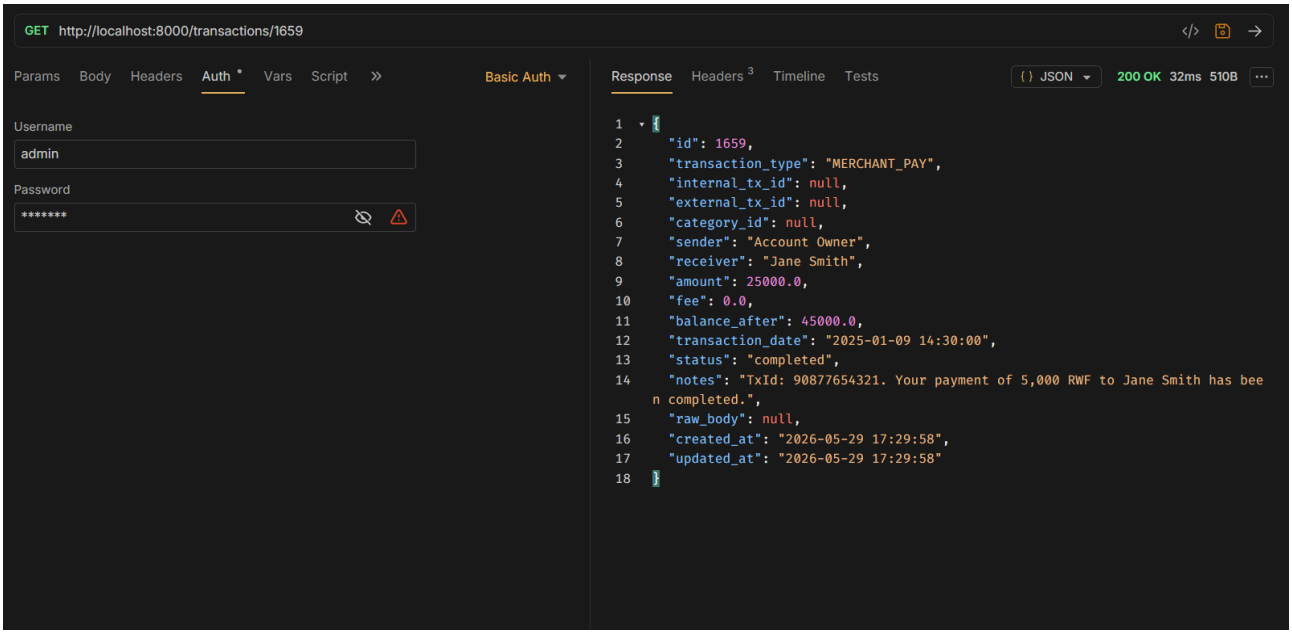


Status Code	Meaning
200 OK	Request successful, transactions returned
401 Unauthorized	Missing or invalid credentials

ii. GET /transactions/{id}

Returns a single transaction record identified by its numeric ID.

curl -u admin:MOMO123 http://localhost:8000/transactions/1659



Status Code	Meaning
200 OK	Transaction found and returned
400 Bad Request	ID is not a valid number
401 Unauthorized	Missing or invalid credentials
404 Not Found	No transaction exists with that ID

iii. POST /transactions

Creates a new transaction and saves it to the database. Returns 201 Created on success.

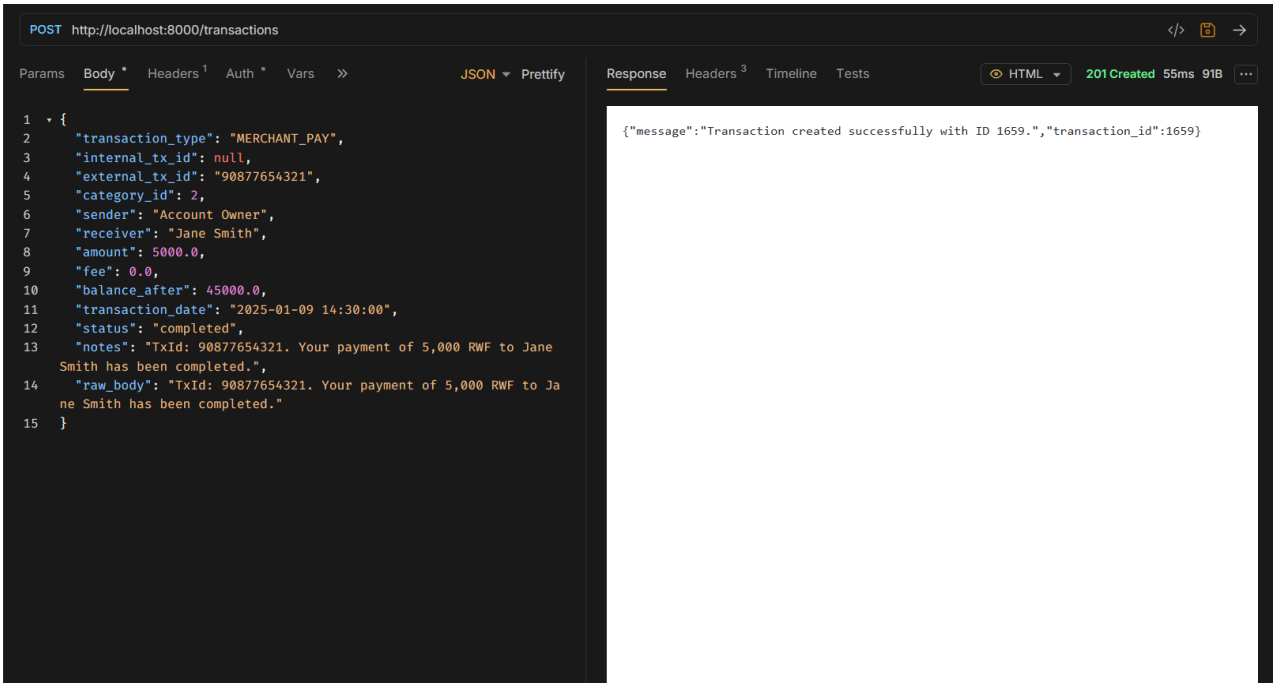
Required fields: transaction_type, amount, sender

Optional fields: external_tx_id, receiver, fee, balance_after, transaction_date, status, notes, raw_body

*curl -u admin:MOMO123 -X POST -H "Content-Type: application/json" *

*-d '{"transaction_type": "MERCHANT_PAY", "amount": 3000, "sender": "Account Owner"}' *

http://localhost:8000/transactions



Status Code	**Meaning**
201 Created	Transaction created successfully
400 Bad Request	Missing required field, invalid amount, invalid fee, or invalid status
401 Unauthorized	Missing or invalid credentials
500 Server Error	Database error during insert

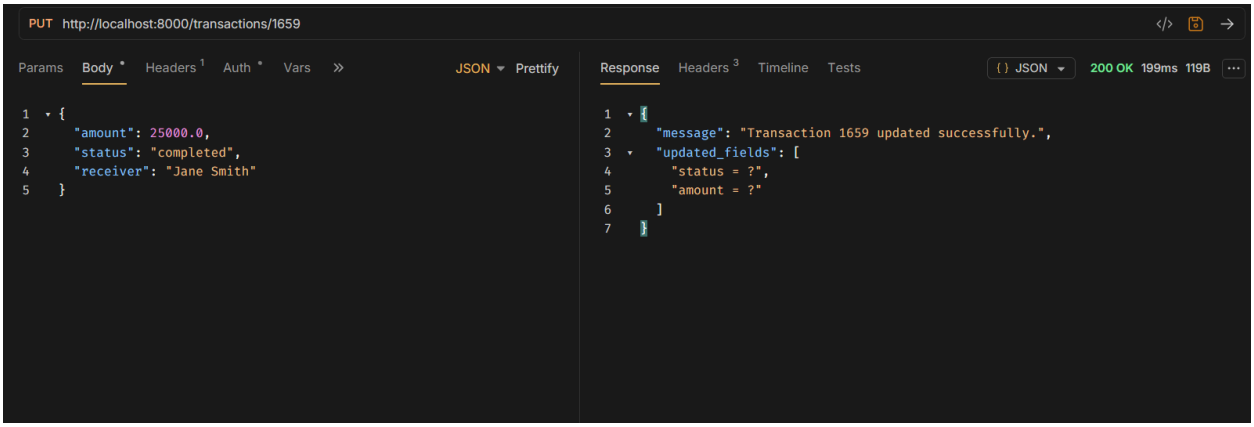
iv. PUT /transactions/{id}

Updates one or more fields of an existing transaction. Only send the fields you want to change.

*curl -u admin:MOMO123 -X PUT -H "Content-Type: application/json" *

*-d '{"status": "reversed"}' *

http://localhost:8000/transactions/1659

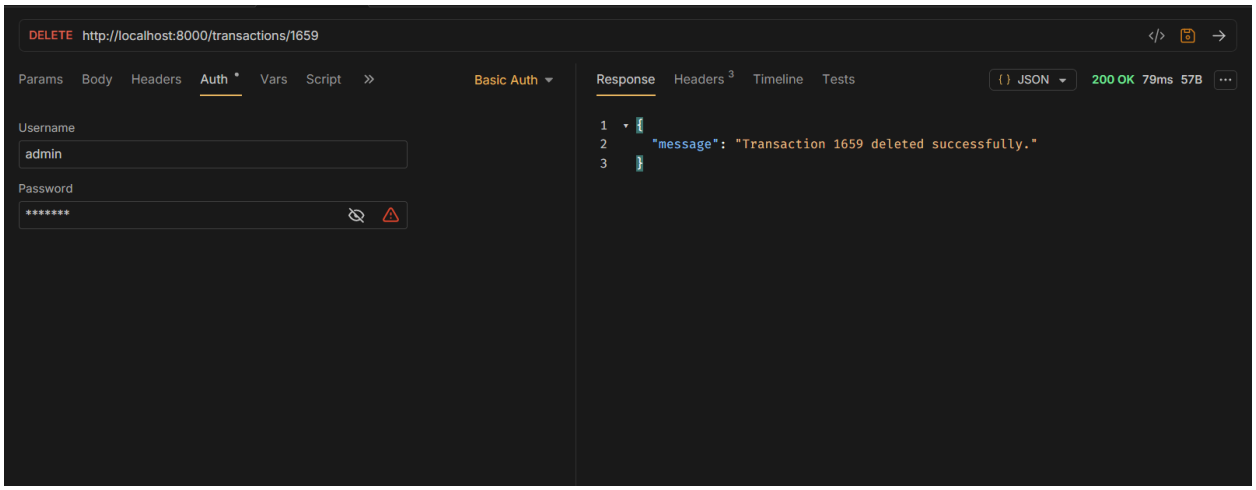


Status Code	Meaning
200 OK	Transaction updated successfully
400 Bad Request	No valid fields provided or invalid JSON
401 Unauthorized	Missing or invalid credentials
404 Not Found	No transaction exists with that ID

v. DELETE /transactions/{id}

Permanently removes a transaction from the database. This action cannot be undone.

curl -u admin:MOMO123 -X DELETE http://localhost:8000/transactions/1659



Status Code	Meaning
200 OK	Transaction deleted successfully
400 Bad Request	ID is not a valid number
401 Unauthorized	Missing or invalid credentials
404 Not Found	No transaction exists with that ID

4. Data Structure and Algorithms - Efficiency Comparison

One of the major requirements for building a good API is choosing the right data structure for searching through the records. In this section we implemented and compared two approaches for finding a transaction by ID using **Linear Search and Dictionary Lookup.**

i. Linear Search

Linear Search scans through every record in the list from the beginning one by one until it finds the one with the matching ID. In the best case the match is the first record. In the worst case the match is the last record or does not exist at all, requiring a full scan of the entire dataset.

Time complexity: This method takes time complexity of O(n) performance to degrade linearly as the dataset grows. With 4,177 transactions, the worst case checks all 4,177 records for every single lookup.

```
def linear_search(transactions, target_id):
    for transaction in transactions:
        if transaction['id'] == target_id:
            return transaction
    return None
```

ii. Dictionary Lookup

Dictionary Lookup builds a dictionary data type that maps each transaction ID directly to its record. These dictionaries use a hash table, when you look up a key, using the python programme language, computes a hash of the key and jumps directly to the memory location where that record is stored. No scanning is required.

Time complexity: This method's time complexity is O(1), lookup time remains the same regardless of the size of the dataset. Looking up one transaction from 4,00 takes the same time as looking up one transaction from 4,000,000.

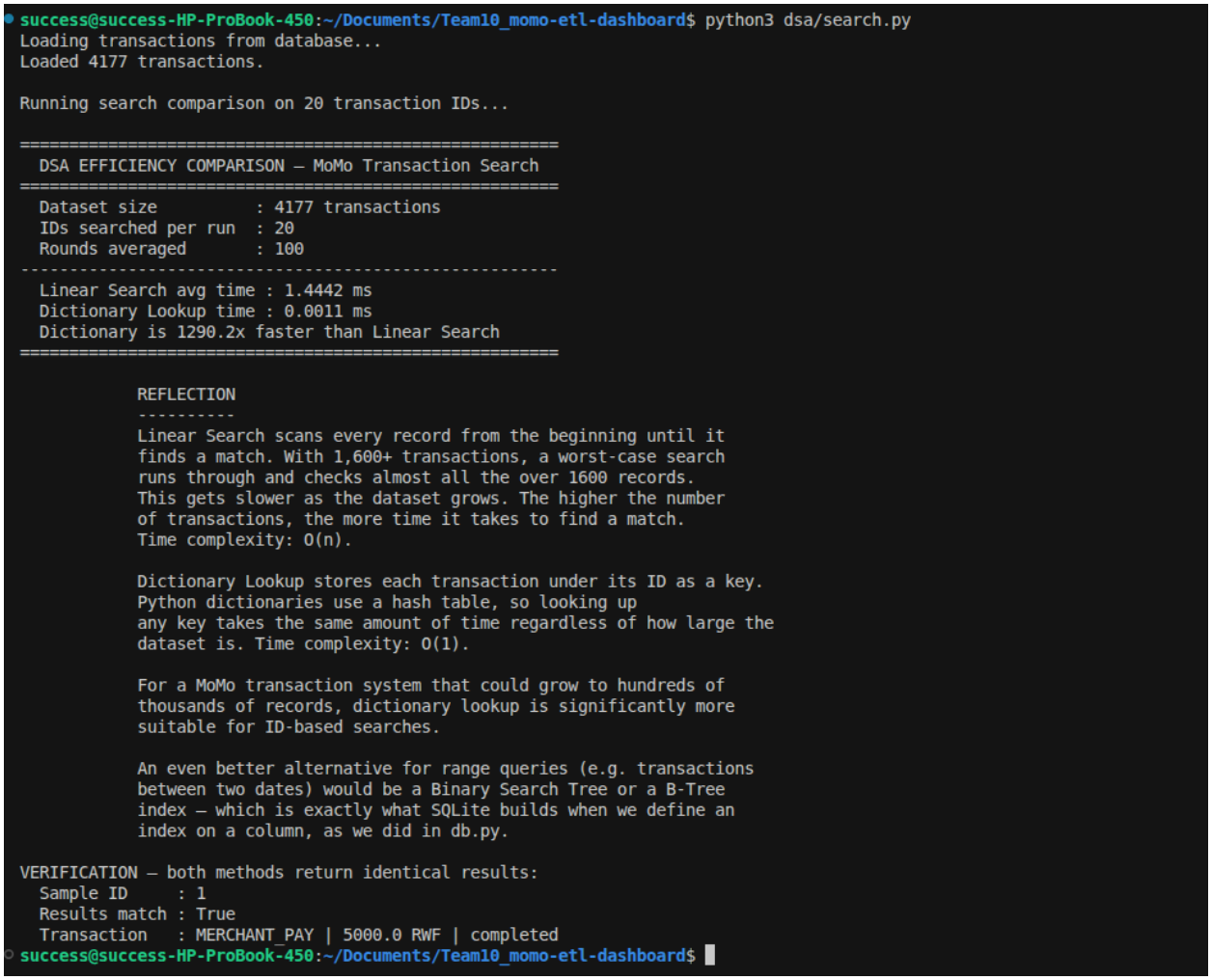
```
def build_lookup_dict(transactions):
    return {transaction['id']: transaction for transaction in transactions}
def dictionary_lookup(lookup_dict, target_id):
```

return lookup_dict.get(target_id, None)

iii. Efficiency Comparison Results

We used both the Linear Search and Dictionary Lookup to test against the full database of 4,177 real MoMo transactions. And had 20 IDs spread evenly across the dataset searched, and each method was run 100 times to produce stable average timing measurements.

Metric	**Linear Search**	**Dictionary Lookup**
Dataset size	4,177 transactions	4,177 transactions
IDs searched per run	20	20
Rounds averaged	100	100
Average time	1.5528 ms	0.0051 ms
Time complexity	O(n)	O(1)
Speed difference	Unknown	1290.2x faster



iv. Reflection

The results show that Dictionary Lookup is about 1290.2x faster than Linear Search. This is just for the data we have here, because based on what we noticed, this gap can grow when more data is added to the set. At 4,177 records Linear Search averages 1.5528 ms and would grow with more data but Dictionary Lookup would remain at approximately 0.005 ms regardless.

The reason for this difference is based on the underlying data structure. A list requires sequential access, to find element N, Python must check elements 1 through N that is index 0 to index N-1. A dictionary uses a hash table where

the key is transformed into a memory address, whereby direct access in a single step.

For the MoMo transaction system, dictionary lookup is the best choice for ID-based searches.

B-Tree index would also be an efficient approach, the data structure that relational databases like SQLite use internally when you create an index on a column. B-Trees allow efficient range queries (find all transactions between two dates) with O(log n) complexity, whereas a hash table only supports exact key lookups. In the database schema built for this project, indexes were placed on transaction_date, status, and amount precisely for this reason.

5. API Testing and Validation

All endpoints were tested using curl from the command line. The following screenshots demonstrate each test case required.

Test Case	**Expected Result**	**Actual Result**
GET /transactions (no credentials)	401 Unauthorized	401 Unauthorized
GET /transactions (wrong password)	401 Unauthorized	401 Unauthorized
GET /transactions (valid credentials)	200 OK + data	200 OK + 4,177 transactions
GET /transactions/1 (valid credentials)	200 OK + record	200 OK + transaction record

Test Case	Expected Result	Actual Result
POST /transactions (valid credentials)	201 Created	201 Created, id=4178
PUT /transactions/4178 (update status)	200 OK	200 OK, updated successfully
DELETE /transactions/4178	200 OK	200 OK, deleted successfully